

CI/CD Pipeline For WebApplication Using AWS

VENKADESH.V

Class Time: 3.30pm to 5.00pm

Joining date: 20/02/2026

INTRODUCTION

The CI/CD Pipeline for Web Application using AWS is a DevOps project developed to automate the process of building, testing, and deploying a web application. The project integrates Git, GitHub, Jenkins, Docker, Python Flask, and AWS EC2 to create an efficient automated deployment workflow. Whenever new code is pushed to GitHub, a webhook automatically triggers Jenkins to execute the pipeline. Jenkins tests the application, builds a Docker image, deploys the application as a container, and verifies its health. This project demonstrates how CI/CD automation can reduce manual effort, improve deployment speed, and provide reliable application delivery.

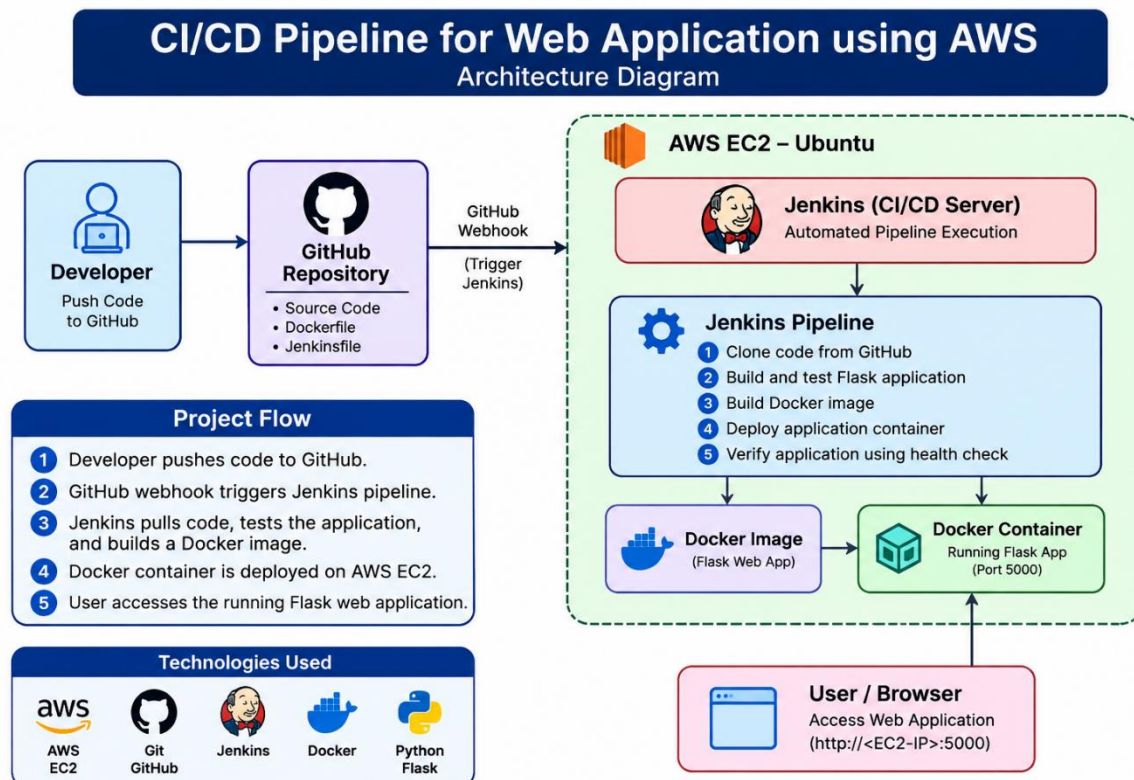
OBJECTIVES

- To automate the application deployment process using CI/CD practices.
- To integrate Git, GitHub, Jenkins, and Docker into a single automated workflow.
- To automatically build, test, and deploy the Flask web application whenever new code is pushed to GitHub.
- To use AWS EC2 as the cloud environment for hosting Jenkins and the Dockerized application.
- To reduce manual effort and improve deployment speed, consistency, and reliability.

TECHNOLOGIES USED

- AWS EC2 – Hosts the entire project
- Ubuntu - Operating system
- Git - Version control
- GitHub - Source code repository
- Jenkins - CI/CD automation
- Docker - Application containerization
- Flask - Web application
- GitHub Webhook - Automatically triggers Jenkins

ARCHITECTURE DIAGRAM



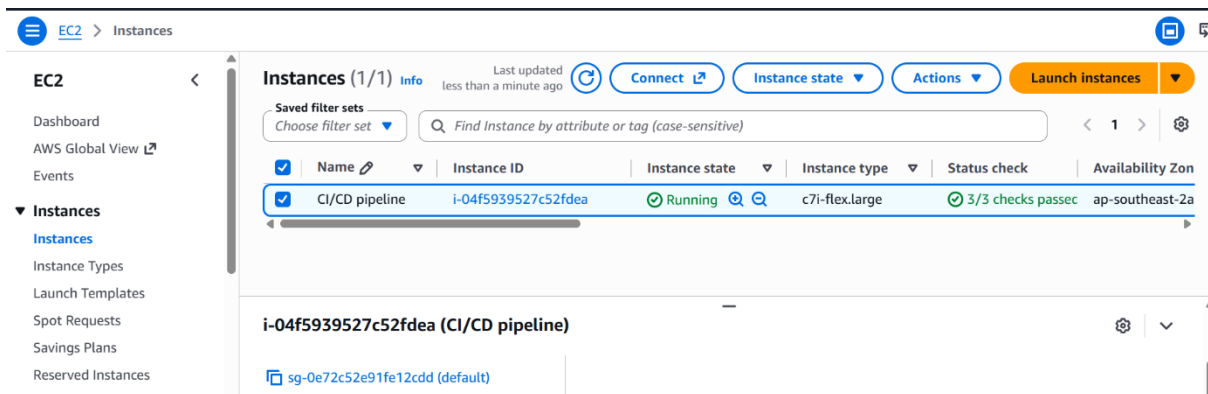
IMPLEMENTATION

AWS EC2 Setup

Open the AWS console and launch the new instance and named it, then connect it to the terminal.

The required Security Group rules were configured to allow:

- SSH Access (Port 22)
- Flask Application (Port 5000)
- Jenkins (Port 8080)
- HTTP (Port 80)



Update the Ubuntu package using the command

- ✓ sudo apt update
- ✓ sudo apt upgrade -y

```
Get:54 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f
Fetched 36.9 MB in 4s (9732 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
137 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-5-112:~$
```

Install Git & Docker

Install the git , because it tracks changes in our application source code.

- `sudo apt install git -y`
- `git --version`

```
ubuntu@ip-172-31-5-112:~$ sudo apt install git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-5-112:~$ git --version
git version 2.43.0
ubuntu@ip-172-31-5-112:~$
```

Install the docker because, Docker packages the Flask application and its dependencies into a container.

- `sudo apt install docker.io -y`
- `sudo systemctl start docker`
- `sudo systemctl enable docker`
- `sudo systemctl status docker`

```
ubuntu@ip-172-31-5-112:~$ sudo systemctl start docker
ubuntu@ip-172-31-5-112:~$ sudo systemctl enable docker
ubuntu@ip-172-31-5-112:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-09-05 05:24:38 UTC; 28s ago
     TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 27465 (dockerd)
       Tasks: 9
      Memory: 23.2M (peak: 23.5M)
         CPU: 192ms
    CGroup: /system.slice/docker.service
            └─27465 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Allow ubuntu user to run the docker without sudo.

- `sudo usermod -aG docker ubuntu`
- `newgrp docker`

Now we checking without the sudo

- `docker ps`

```
ubuntu@ip-172-31-5-112:~$ sudo usermod -aG docker ubuntu
ubuntu@ip-172-31-5-112:~$ newgrp docker
ubuntu@ip-172-31-5-112:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-5-112:~$
```

Create Web Application & Dockerfile

Create the project folder:

- `mkdir project`

Enter the directory

- `Cd project`

```
ubuntu@ip-172-31-5-112:~$ ls
project
ubuntu@ip-172-31-5-112:~$ cd project
ubuntu@ip-172-31-5-112:~/project$ ls
Dockerfile  Jenkinsfile  app.py  requirements.txt
```

Now Build the docker image using

- `docker build -t devops-webapp .`
- `docker images`

now we can see the output

```

Step 6/7 : EXPOSE 5000
--> Running in f6891dacbbe3
--> Removed intermediate container f6891dacbbe3
--> f7b7731a65bd
Step 7/7 : CMD ["python", "app.py"]
--> Running in ela4363b05dd
--> Removed intermediate container ela4363b05dd
--> 795e175d23c6
Successfully built 795e175d23c6
Successfully tagged devops-webapp:latest
ubuntu@ip-172-31-5-112:~/project$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
devops-webapp:latest	795e175d23c6	208MB	51.2MB	
python:3.12-slim	78387bc3881b	190MB	48.4MB	

```

ubuntu@ip-172-31-5-112:~/project$

```

Now Run the container using

- `docker run -d --name devops-webapp -p 5000:5000 devops-webapp`
- `docker ps`

```

ubuntu@ip-172-31-5-112:~/project$ docker run -d --name devops-webapp -p 5000:5000 devops-webapp
d91f87dbc1a986cde3fa8a4d3820dbc770044443cd3be7cf07b58c6708f9836d
ubuntu@ip-172-31-5-112:~/project$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d91f87dbc1a9	devops-webapp	"python app.py"	10 seconds ago	Up 10 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	devops-webapp

```

ubuntu@ip-172-31-5-112:~/project$

```

Now test the application In terminal using

- `curl http://localhost:5000`
- `curl http://localhost:5000/health`

```

ubuntu@ip-172-31-5-112:~/project$ curl http://localhost:5000

```

```

<html>
  <head>
    <title>DevOps CI/CD Project</title>
  </head>
  <body>
    <h1>Welcome to DevOps CI/CD Pipeline</h1>
    <h2>Application is Running Successfully</h2>
    <p>Git → GitHub → Jenkins → Docker → AWS EC2</p>
    <p>Monitoring: Prometheus → Grafana</p>
  </body>
</html>
ubuntu@ip-172-31-5-112:~/project$ curl http://localhost:5000/health
{"application":"devops-webapp","status":"healthy"}
ubuntu@ip-172-31-5-112:~/project$

```

Now open the web browser using your EC2 public ip to see the application dashboard

- `http://YOUR_EC2_PUBLIC_IP`

Welcome to DevOps CI/CD Pipeline

Application is Running Successfully

Git → GitHub → Jenkins → Docker → AWS EC2

Monitoring: Prometheus → Grafana

Push the project to Github

Configure Git on EC2

- `git config --global user.name "username"`
- `git config --global user.email "YOUR_GITHUB_EMAIL"`
- `git config --global --list`

```
ubuntu@ip-172-31-5-112:~$ git config --global --list
user.name=Venkadesh42
user.email=venkadesh05111999@gmail.com
ubuntu@ip-172-31-5-112:~$
```

Initiate the git

- `git init`

```
ubuntu@ip-172-31-5-112:~/project$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/ubuntu/project/.git/
ubuntu@ip-172-31-5-112:~/project$
```

- `git add .`

- git status

```
ubuntu@ip-172-31-5-112:~/project$ git add .
ubuntu@ip-172-31-5-112:~/project$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   Dockerfile
        new file:   app.py
        new file:   requirements.txt

ubuntu@ip-172-31-5-112:~/project$
```

Now create a github repository to store code

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#)
Required fields are marked with an asterisk (*).

1 General

Owner * venkadesh42 / Repository name *

✔ Your new repository will be created as ci-cd-pipeline-. The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. How about [redesigned-potato?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Now ready to push the code to repository

- git commit -m "comment"
- git branch -M main
- git remote add origin
https://github.com/YOUR_USERNAME/devops-cicd-project.git
- git push -u origin main

```

ubuntu@ip-172-31-5-112:~/project$ git add .
ubuntu@ip-172-31-5-112:~/project$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   Dockerfile
        new file:   app.py
        new file:   requirements.txt

ubuntu@ip-172-31-5-112:~/project$

```

```

ubuntu@ip-172-31-5-112:~/project$ git remote add origin https://github.com/venkadesh42/ci-cd-pipeline-.git
ubuntu@ip-172-31-5-112:~/project$ git remote -v
origin  https://github.com/venkadesh42/ci-cd-pipeline-.git (fetch)
origin  https://github.com/venkadesh42/ci-cd-pipeline-.git (push)
ubuntu@ip-172-31-5-112:~/project$ git push -u origin main
Username for 'https://github.com': venkadesh42
Password for 'https://venkadesh42@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 895 bytes | 895.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/venkadesh42/ci-cd-pipeline-.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
ubuntu@ip-172-31-5-112:~/project$

```

now we can see the files are moved to guthub

The screenshot shows the GitHub interface for a repository named 'ci-cd-pipeline-' owned by 'venkadesh42'. The repository is public and has 1 branch (main) and 0 tags. It shows 4 commits by user 'edacad6' from yesterday. The commit history table lists the following files and their descriptions:

File	Description	Commit Date
.gitignore	Initial web application	yesterday
Dockerfile	Initial web application	yesterday
Jenkinsfile	Add Jenkins CI/CD pipeline	yesterday
app.py	Test automatic CI/CD deployment	yesterday
requirements.txt	Test automatic CI/CD deployment	yesterday

Below the commit history is a 'README' section. On the right side of the repository page, there is an 'About' section with the text 'No description, website, or topics provided.' and statistics: 0 stars, 0 watching, and 0 forks. There is also a 'Releases' section stating 'No releases published' and a link to 'Create a new release'. At the bottom, there is a 'Packages' section.

Install Jenkins

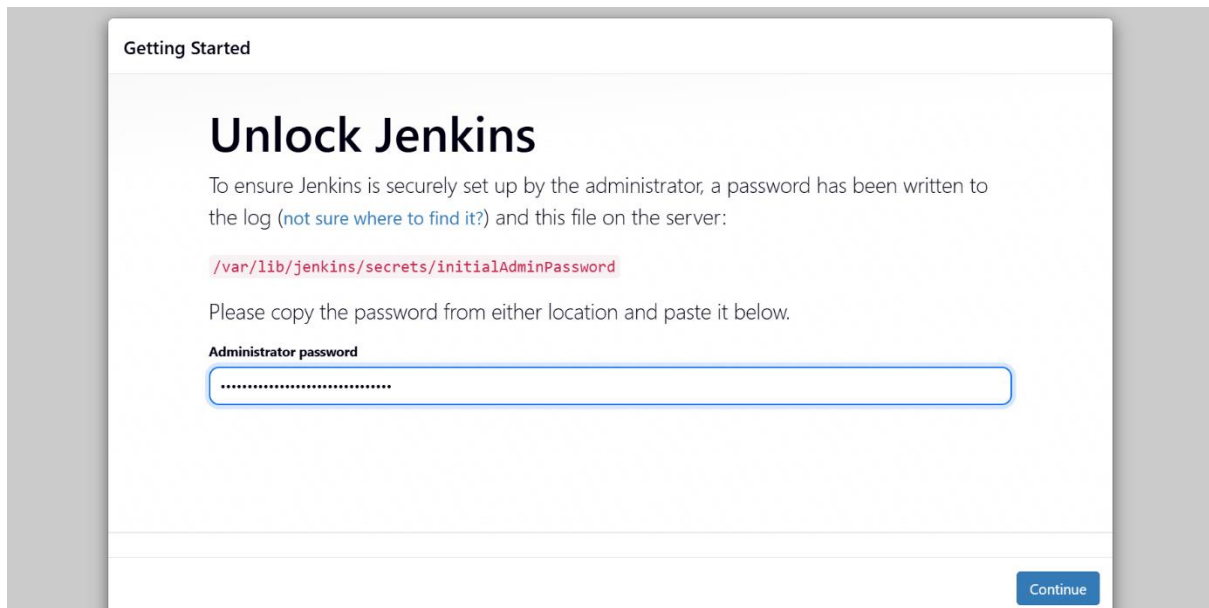
Install Jenkins using the commands also java,Jenkins repository

- `sudo apt install fontconfig openjdk-21-jre -y`
- `sudo mkdir -p /etc/apt/keyrings`
- `sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \`
- `https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key`
- `echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null`
- `sudo apt install jenkins -y`
- `sudo systemctl enable jenkins`
- `sudo systemctl start Jenkins`
- `sudo systemctl status Jenkins`

```
sudo systemctl start jenkins
Synchronizing state of jenkins.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable jenkins
ubuntu@ip-172-31-5-112:~/project$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-09-05 05:56:21 UTC; 20s ago
     Main PID: 31189 (java)
       Tasks: 50 (limit: 4512)
      Memory: 716.6M (peak: 722.7M)
         CPU: 14.102s
    CGroup: /system.slice/jenkins.service
            └─31189 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --

Sep 05 05:56:16 ip-172-31-5-112 jenkins[31189]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
```

- `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
use this command to get a password to login Jenkins.



We give GitHub credentials to Jenkins so Jenkins can access the GitHub repository securely.



Create a new job and give name and select git under the scm and select the credentials then click save.

Then click the build now to see the process of build, test and deploy.

Jenkins / cicd pipeline / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/venkadesh42/ci-cd-pipeline-.git

Credentials ?

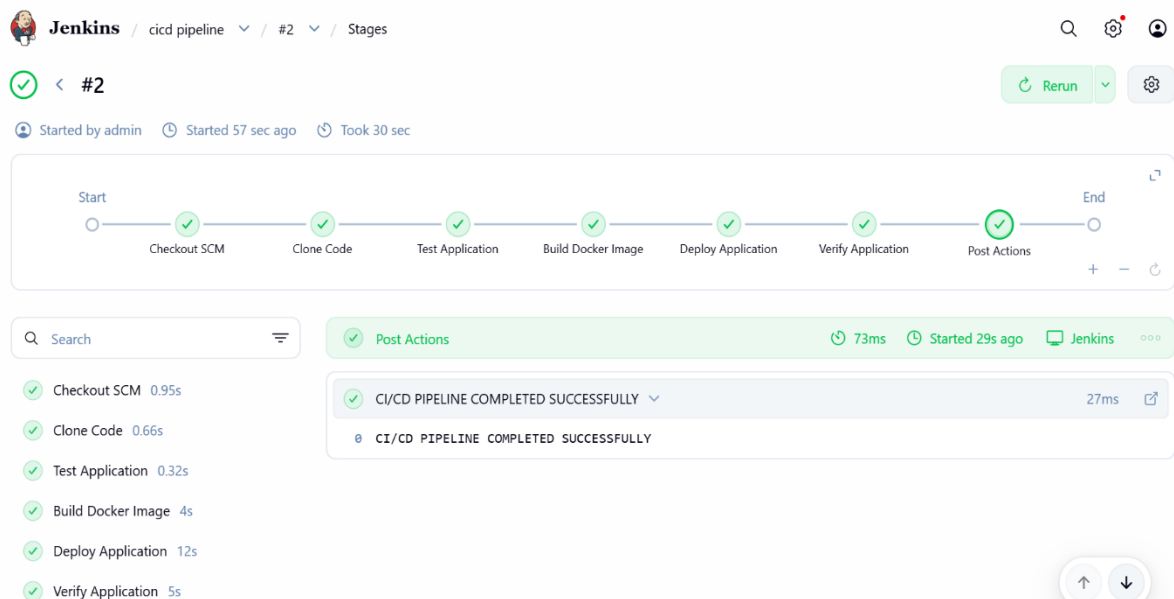
venkadesh42/* (github)

+ Add

Advanced

+ Add Repository

Save Apply



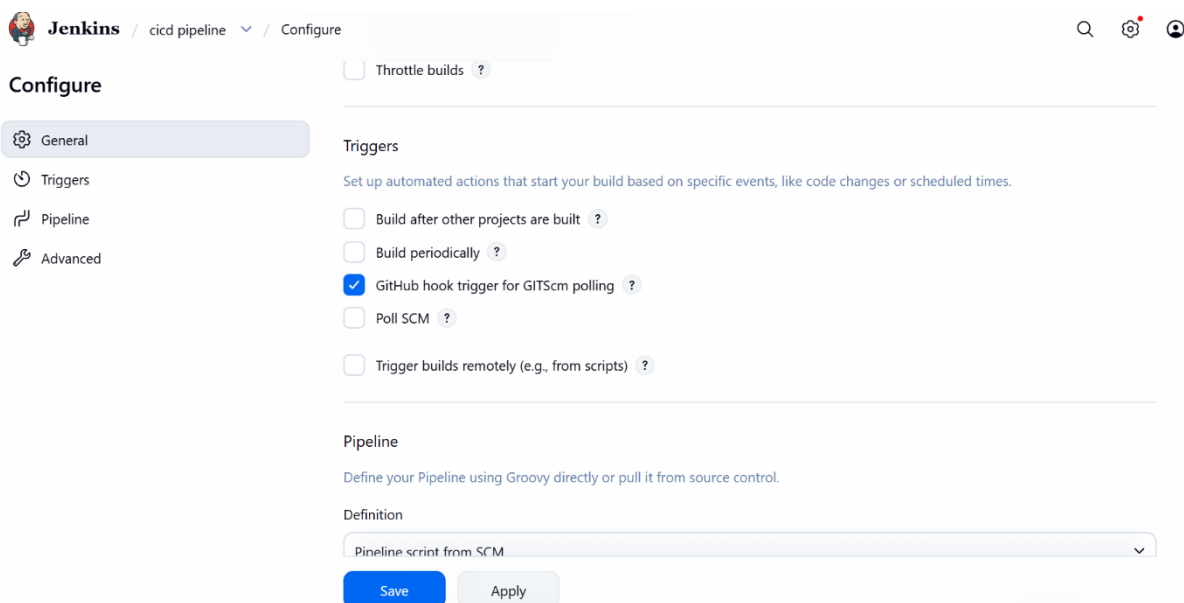
Now check the application using docker ps

```
buntu@ip-172-31-5-112:~/project$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
1361a68b5d8   devops-webapp:latest  "python app.py"         About a minute ago    Up About a minute    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
buntu@ip-172-31-5-112:~/project$
```

Automatic Trigger From Jenkins To Github

Adding github webhook to automaticall trigger and run the test when code is pushed to github.

Find build trigger and enable github webhook in Jenkins.



Add this link in webhook to connect Jenkins to automate the testing process .

<http://3.25.215.120:8080/github-webhook/>

now again we check the process how its work after adding webhook for automate process.

This is the old dashboard.

Welcome to DevOps CI/CD Pipeline

Application is Running Successfully

Git → GitHub → Jenkins → Docker → AWS EC2

Monitoring: Prometheus → Grafana

Here you can see application is running successfully in old dashboard.

After some change now new code is pushed to github.

```
ubuntu@ip-172-31-5-112:~/project$ nano app.py
ubuntu@ip-172-31-5-112:~/project$ git add .
git commit -m "Update application version 2.0"
git push
[main 466a954] Update application version 2.0
 1 file changed, 1 insertion(+), 1 deletion(-)
Username for 'https://github.com': venkadesh42
Password for 'https://venkadesh42@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 317 bytes | 317.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/venkadesh42/ci-cd-pipeline-.git
   e2d4f41..466a954  main -> main
ubuntu@ip-172-31-5-112:~/project$
```

After this now Jenkins pipeline build automatically without the manual start because when the new code is pushed so the webhook trigger the Jenkins to start the process.

After this you can see the changes in web dashboard also.

The screenshot shows the Jenkins web interface for a pipeline named 'cicd pipeline'. The top navigation bar includes the Jenkins logo, the pipeline name, and search, settings, and user icons. A left sidebar contains links: 'Build Now', 'Delete Pipeline', 'Configure', 'Rename', 'Stages', 'Pipeline Syntax', and 'GitHub Hook Log'. The main area displays build history with a list of actions: 'Last build (#2), 4 min 54 sec ago', 'Last stable build (#2), 4 min 54 sec ago', 'Last successful build (#2), 4 min 54 sec ago', and 'Last completed build (#2), 4 min 54 sec ago'. Below this, a 'Builds' dropdown menu is open, showing a search filter and a list of builds for 'Today': build #3 at 6:28 AM (with a progress bar and status icons), build #2 at 6:17 AM, and build #1 at 6:12 AM.

Welcome to DevOps CI/CD Pipeline

Application version 2.0

Git → GitHub → Jenkins → Docker → AWS EC2

Monitoring: Prometheus → Grafana

Now you can see the corrections, so the ci/cd pipeline web application is working correctly without manual start using automatically by webhook.

CONCLUSION

The CI/CD Pipeline for Web Application using AWS successfully demonstrates the automation of the software development and deployment process. The integration of

GitHub, Jenkins, Docker, and AWS EC2 enables the application to be tested, built, and deployed automatically. The pipeline also performs a health check to ensure that the application is running successfully after deployment. Overall, this project demonstrates how DevOps and CI/CD practices can improve deployment efficiency, reliability, and consistency.